Runtime Analysis

We assume there are n courses stored in the data structure. The following analysis describes the worst-case runtime for loading the file and creating course objects. This analysis does not include the menu loop, printing, and or searching options.

Operation	Vector	Hash Table	Binary Search Tree
Read & parse file	O(n)	O(n)	O(n)
Validate prerequisites	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insert courses into structure	O(n)	O(n) average	$O(n^2)$ worst
Overall worst-case load time	$O(n^2)$	$O(n^2)$	$O(n^2)$

Advantages and Disadvantages

Vector

A vector is simple to implement and easy to iterate through but searching for a single course is a linear operation in the worst case, and printing courses in alphanumeric order requires sorting the vector first.

Hash Table

A hash table supports fast average case insertion and lookup for printing a single course, but it does not maintain sorted order, so printing the full course list requires collecting and sorting keys before printing.

Binary Search Tree

A BST supports printing courses in alphanumeric order using an in-order traversal and supports efficient searching in average cases. However, if the tree becomes unbalanced, insertion and searching can degrade toward linear performance.

Recommendation

Based on the requirements to print all courses in alphanumeric order and print a specific course with prerequisites, I recommend using the Binary Search Tree. It supports printing the complete course list in sorted order through an in-order traversal without requiring a separate sorting step, and it provides efficient searching in average cases compared to a vector.